โปรเจคโปรแกรมเตือน/ล็อคจอ ตามเวลาจ้องจอ — เอกสารไอเดีย

> สรุปจากที่คุยกัน: Windows + Python, วัดจาก input/idle ผสมกับ active window, เตือนล่วงหน้าก่อนล็อค, ข้ามได้แต่มีต้นทุน, กติกา 20-20-20, มี Dashboard สถิติ, Whitelist/DND, Tray + autostart, หน้าตั้งค่า, พักแบบมีกิจกรรมให้ทำ

---

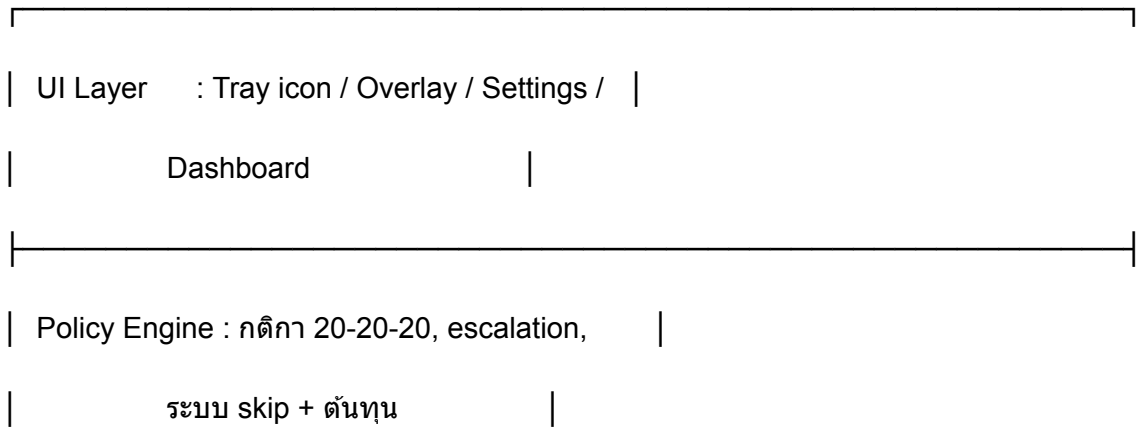
1. ชื่อโปรเจค (เลือกไว้ใช้ใน portfolio)

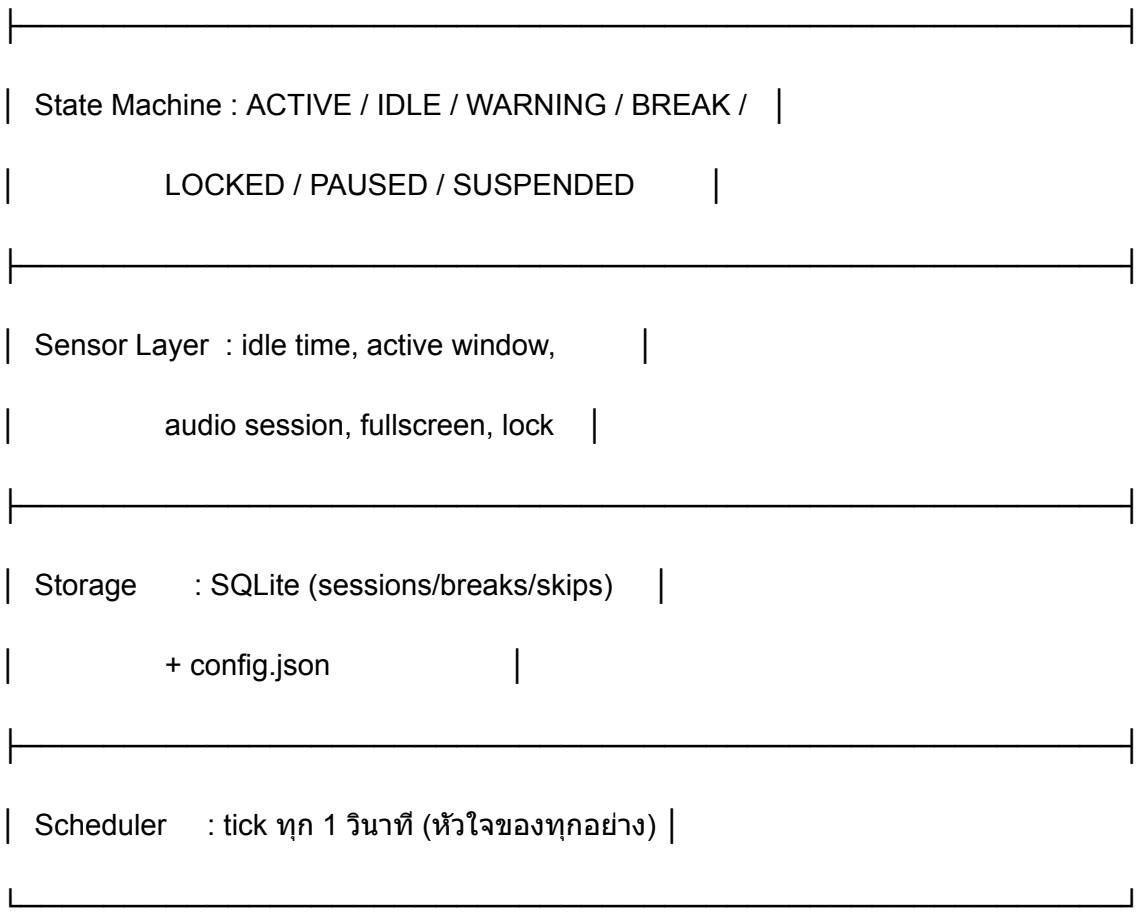
ชื่อ	ที่มา
Blink	สั้น จำง่าย ตรงกับเรื่องกระพริบตา
20-20-20 / Twenty	บอกกติกาตรงๆ คนสายสุขภาพเข้าใจทันที
Farsight	เล่นคำ "มองไกล" + "มองการณ์ไกล"
RestEye / EyeGuard	ตรงไปตรงมา หา SEO ง่าย
Ma-Rest (มาเรส)	เล่นคำไทย ถ้าอยากทำตลาดไทย

---

2. ภาพรวมสถาปัตยกรรม

แยกเป็น 6 ชั้น อย่าเขียนรวมกันเป็นไฟล์เดียว เพราะ Dashboard กับ Overlay จะพันกันมั่ว





หลักคิดสำคัญ: ทุกอย่างขับด้วย tick ละ 1 วินาที ที่อ่านค่า sensor → อัปเดต state → ถาม policy ว่าต้องทำอะไร → สั่ง UI ห้ามให้ UI มีตรรกะการนับเวลาเอง เพราะจะ debug ไม่ได้เลย

---

3. Sensor Layer — วิธีวัดว่า "จ้องจออยู่จริง"

นี่คือส่วนที่ยากที่สุดและเป็นจุดที่โปรเจคจะน่าสนใจ

3.1 Idle time (แกนหลัก)

ใช้ Win32 API `GetLastInputInfo` ผ่าน `ctypes` — คืนค่าว่า "ผ่านไปกี่ ms ตั้งแต่มี เมาส์/คีย์บอร์ด ขยับล่าสุด"

ทำไมต้องใช้ตัวนี้ ไม่ใช่ keyboard hook:

- ไม่ต้องดักปุ่มที่กด → **ไม่ใช่ keylogger** → antivirus ไม่ตีตก และเขียนใน README ได้ว่า privacy-safe by design (จุดขายสำหรับ portfolio)
- ไม่ต้องรัน admin

- กินทรัพยากรแทบเป็นศูนย์

3.2 Active window (แกนเสริม)

`GetForegroundWindow()` → `GetWindowThreadProcessId()` → `psutil` เอาชื่อ process

+ `GetWindowText()` เอา title

ได้ 3 อย่าง:

1. รู้ว่าใช้แอปไหนอยู่ → ใส่ใน Dashboard
2. เช็ค whitelist ได้
3. แยกงานกับพักผ่อนได้

3.3 ปัญหาคลาสสิก: "ดูวิดีโออยู่ ไม่ขยับเมาส์ แต่จ้องจอเต็มๆ"

ถ้าใช้ idle อย่างเดียว โปรแกรมจะคิดว่าคุณพักแล้ว ทั้งที่กำลังทำร้ายตามากที่สุด วิธีเดา (ใช้ร่วมกันเป็น heuristic ให้คะแนน):

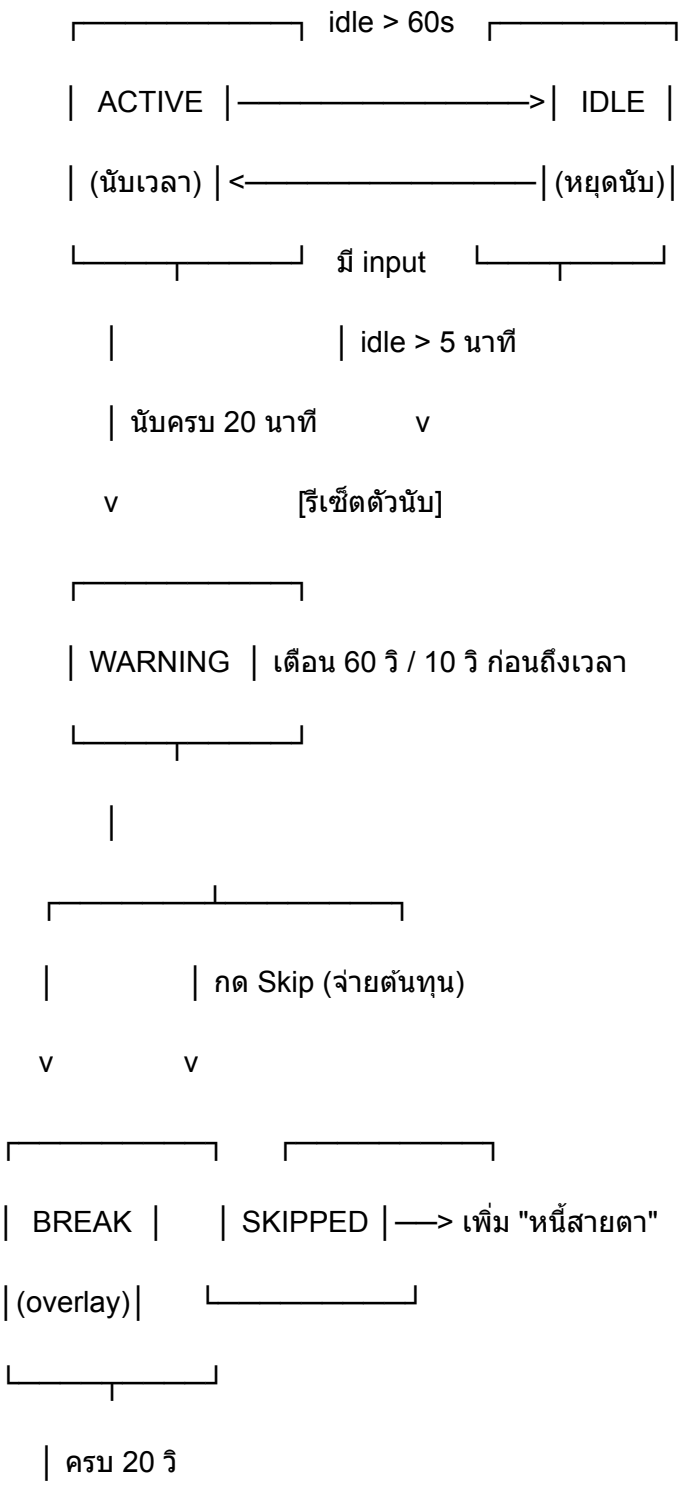
สัญญาณ	**วิธีเช็ค**	**บอกอะไร**
มีเสียงเล่นอยู่	`pycaw` → AudioSession state == Active	น่าจะดูวิดีโอ/ประชุม
หน้าต่าง fullscreen	เทียบ rect ของ foreground window กับ rect ของจอ	ดูหนัง/เล่นเกม
แอปกันจอดับ	`powercfg /requests` (ต้อง admin) หรือดูว่าจอไม่หรี่ทั้งที่ idle	มีสื่อเล่นอยู่แน่
ชื่อ process	เทียบกับลิสต์ (vlc, mpv, chrome+title มี YouTube)	ตรงตัวที่สุด แต่ต้อง maintain

กติกาที่แนะนำ: ถือว่า "กำลังจ้องจอ" ถ้า `idle < 60 วินาที` **หรือ** `(มีเสียงเล่น และ หน้าจอไม่ได้ล็อค)`

3.4 กรณีที่ต้อง "หยุดนับ" ให้ถูก (พลาดตรงนี้แล้วโปรแกรมจะน่ารำคาญมาก)

- **จอล็อคอยู่** — ตรวจด้วย `WTSRegisterSessionNotification` หรือวิธีง่ายกว่าคือดูว่า `OpenInputDesktop()` คืน None
- **เครื่อง sleep / hibernate** — วิธีดักง่ายๆ: เก็บ timestamp ของ tick ก่อนหน้า ถ้า tick นี้ห่างจากอันก่อนเกิน ~30 วินาที แปลว่าเครื่องหลับไป → ถือเป็นการพัก (เทคนิคนี้ครอบคลุมเคสแปลกๆ ได้เยอะโดยไม่ต้องเขียน API เพิ่ม)
- **idle นานเกิน 5 นาที** → รีเซ็ตตัวนับสายตา เพราะถือว่าลุกไปพักจริงแล้ว

4. State Machine



v

[รีเซ็ต + บันทึกลง DB]

เพิ่ม PAUSED (ผู้ใช้สั่งพักโปรแกรมเอง) และ SUSPENDED (whitelist/DND ทำงานอยู่)

---

5. Policy Engine — 20-20-20 + การเตือนแบบไล่ระดับ

คุณเลือก "เตือนก่อนล่วงหน้า" ซึ่งถูกมาก เพราะสิ่งที่ทำให้แอปแนวนี้โดนถอนคือ **การขัดจังหวะแบบไม่ทันตั้งตัว** ตอนกำลังพิมพ์อยู่

Escalation ladder ที่แนะนำ

เวลา	สิ่งที่เกิดขึ้น	ระดับการรบกวน
นาทีที่ 18	Toast มุมขวาล่าง "อีก 2 นาทีพักสายตา"	ต่ำ — ปิดเองได้
นาทีที่ 19:50	แถบบางๆ ขอบจอค่อยๆ เปลี่ยนสี / จอหรี่ลง 10%	กลาง — เห็นด้วยหางตา
นาทีที่ 20:00	Overlay เต็มจอ นับถอยหลัง 20 วินาที	สูง — บังคับ
หลัง overlay	บันทึกลง DB, รีเซ็ตตัวนับ, ตัวจับเวลาเริ่มใหม่	—

แนวคิด "Grace period" (สำคัญมาก)

ถ้าตอนครบ 20 นาทีผู้ใช้กำลัง**พิมพ์รัวอยู่** ให้เลื่อนออกไปได้สูงสุด 2 นาที รอจังหวะที่ idle สัก 3 วินาทีแล้วค่อยเด้ง — ความรู้สึกต่างกันคนละโลก (วัดจาก "มี input ในช่วง 5 วินาทีล่าสุดกี่ครั้ง")

---

6. ระบบ Skip แบบมีต้นทุน — หัวใจของโปรเจค

นี่คือส่วนที่จะทำให้โปรเจคนี้ต่างจาก "แอปเตือนพักสายตา" ทั่วไป ไอเดียเรียงจากอ่อนไปแข็ง เลือกผสมได้:

6.1 ต้นทุนแบบ Friction (ทำให้ช้าลง)

- **กดค้าง 5 วินาที** ถึงจะข้ามได้ — บังคับให้หยุดคิด ทำง่ายที่สุดและได้ผลดี

- **พิมพ์ประโยคยืนยัน** เช่น "ฉันเลือกที่จะทำร้ายสายตาตัวเอง" — ได้ผลแรง แต่ใช้บ่อยแล้วน่ารำคาญ เหมาะกับ skip ครั้งที่ 3 ขึ้นไปของวัน
- **หน่วงเวลา 10 วินาทีก่อนปุ่มจะกดได้** — ถ้าจะรอ 10 วิอยู่แล้ว พักซะเลยก็ได้

6.2 ต้นทุนแบบ Budget (จำกัดจำนวน)

- ให้ **skip token วันละ 3 อัน** ใช้หมดแล้วหมดเลย
- โชว์จำนวนที่เหลือบน tray icon → เห็นตลอดเวลา
- ทำให้ผู้ใช้ "เก็บไว้ใช้ตอนจำเป็นจริงๆ" ซึ่งคือพฤติกรรมที่เราต้องการพอดี

6.3 ต้นทุนแบบ Debt (แนะนำที่สุด — และเป็นไอเดียที่จะทำให้ portfolio โดด)

"หนี้สายตา" (Eye Debt) — ข้ามได้ แต่หนี้สะสม

- ข้าม 1 ครั้ง = เป็นหนี้ 20 วินาที
- รอบพักถัดไปยาวขึ้นเท่ากับหนี้ที่ค้าง (20 + 20 = 40 วินาที)
- หนี้สะสมได้สูงสุด 3 ครั้ง (เพดาน 80 วินาที) กันไม่ให้บานปลายจนแอปกลายเป็นศัตรู
- หนี้ค้างเกิน 2 ชั่วโมง → บังคับพักยาว ล็อคจอจริง 5 นาที

ข้อดี: ไม่ห้ามผู้ใช้ (เขายังคุมชีวิตตัวเองได้) แต่มีผลจริงเสมอ → ตรงตามหลักพฤติกรรมศาสตร์เรื่อง "delayed cost" และเขียนอธิบายใน README ได้สวยมาก

6.4 ต้นทุนแบบ Social/Record

- ทุกครั้งที่ skip ถูกบันทึกและโชว์เด่นๆ ใน Dashboard เป็นสีแดง
- ทำลาย streak "วันที่พักครบ" ที่สะสมมา
- ทำให้ตัวเลข compliance rate ตก — คนที่ชอบตัวเลขสวยจะเจ็บ

---

7. หน้าจอพัก (Break Overlay) — พักแบบมีกิจกรรม

อย่าให้เป็นจอดำเปล่าๆ เพราะคนจะแค่นั่งจ้องจอดำ ซึ่งไม่ได้พักสายตาเลย

กิจกรรมที่แนะนำ (สุ่มสลับกันไป ไม่ให้เบื่อ)

1. **จุดเคลื่อนที่ให้ตามมอง** — วงกลมค่อยๆ เคลื่อนจากขอบจอด้านหนึ่งไปอีกด้าน บังคับให้กล้ามเนื้อตาทำงานคนละแบบกับตอนจ้องจุดเดียว
2. **นับกระพริบตา** — "กระพริบตาช้าๆ 10 ครั้ง" มีตัวนับใหญ่ๆ ให้กดตาม (คนจ้องจอกระพริบตาน้อยลง ~60% เป็นสาเหตุตาแห้งอันดับหนึ่ง)
3. **มองไกล 20 ฟุต** — ข้อความใหญ่ + วงกลมหดตัวเป็นตัวนับถอยหลัง ไม่มีอะไรให้อ่าน (จงใจให้จอ "น่าเบื่อ" จนอยากหันไปมองที่อื่น — นี่คือดีไซน์ที่ถูกต้อง)
4. **ยืดคอ/ไหล่** — ภาพประกอบง่ายๆ กับคำสั่งสั้นๆ
5. **หายใจ 4-7-8** — วงกลมขยาย/หดตามจังหวะหายใจ

รายละเอียด UX ที่ห้ามลืม

- **ต้องขึ้นทุกจอ** ถ้าใช้หลายมอนิเตอร์ — ไม่งั้นย้ายตาไปจออีกตัวได้เลย
- **Fade in 0.5 วินาที** อย่าเด้งทันที จะตกใจ
- **ปุ่ม Skip ซ่อนอยู่มุมล่าง สีจาง** — หาเจอถ้าตั้งใจหา แต่ไม่เชิญชวน
- **เสียงเบาๆ ตอนเริ่มและจบ** ให้รู้โดยไม่ต้องมอง
- **กด Esc ไม่ควรปิดได้** ให้ปิดได้ทางเดียวคือปุ่ม Skip ที่มีต้นทุน
- **นาฬิกาจริงมุมจอ** เพราะบางคนพักแล้วอยากรู้ว่ากี่โมง

---

8. Whitelist / โหมดห้ามรบกวน

กติกาที่ควรมี (เรียงตามความสำคัญ):

กติกา	ตัวอย่าง	พฤติกรรม
ตามชื่อ process	Zoom, Teams, Discord (ตอนแชร์จอ)	เลื่อนพัก ไม่ยกเลิก
ตาม window title	title มีคำว่า "Presentation" / "Slide Show"	ไม่ขึ้น overlay
Fullscreen exclusive	เกม	ตรวจแล้วเลื่อนออกไป (overlay ขึ้นไม่ได้อยู่ดี)
DND ชั่วคราว	คลิก tray → "หยุด 1 ชม."	หยุดทั้งระบบชั่วคราว
ตามช่วงเวลา	20:00–22:00 ไม่รบกวน	ตั้งใน settings

หมายเหตุสำคัญ: เวลาอยู่ในโหมด whitelist ให้ **"เลื่อน" ไม่ใช่ "ยกเลิก"** เวลาที่จ้องจอยังต้องถูกนับต่อ พอออกจาก Zoom ปุ๊บ ควรเด้งพักทันทีถ้าเกินเวลาแล้ว (และควรมีเพดาน เช่น เลื่อนได้สูงสุด 45 นาที ไม่งั้นเปิด Zoom ค้างไว้ทั้งวันก็โกงได้)

---

9. ฐานข้อมูล & Dashboard

Schema ที่แนะนำ (SQLite — มากับ Python อยู่แล้ว ไม่ต้องลง)

```
-- ช่วงเวลาที่จ้องจอต่อเนื่อง
```

screen_sessions(id, started_at, ended_at, duration_sec, end_reason)

-- end_reason: 'break' | 'idle' | 'lock' | 'shutdown'

-- เหตุการณ์การพัก

breaks(id, scheduled_at, started_at, completed_at, planned_sec,

actual_sec, outcome, debt_applied_sec)

-- outcome: 'completed' | 'skipped' | 'deferred'

-- การใช้งานแอป (sample ทุก 1 นาที ก็พอ ไม่ต้องทุกวินาที)

app_samples(ts, process_name, window_title_hash, is_fullscreen, has_audio)

-- สรุปรายวัน (คำนวณล่วงหน้าไว้ กราฟจะเร็วมาก)

daily_stats(date, total_screen_sec, breaks_completed, breaks_skipped,

longest_session_sec, compliance_rate)

เก็บ **hash ของ window title** แทนตัว title จริง จะปลอดภัยกว่ามาก (title มีข้อมูลส่วนตัวเยอะ เช่น ชื่อไฟล์ ชื่อคน) — และเขียนใน README ได้ว่าออกแบบมาแบบนี้

กราฟที่คุ้มค่าที่สุดใน Dashboard

1. **Heatmap 24 ชม. × 7 วัน** — เห็นทันทีว่าคุณจ้องจอหนักช่วงไหน อันนี้ว้าวที่สุด
2. **Compliance rate รายวัน** — เส้นกราฟ % ที่พักครบ มีเป้าหมาย 80% ขีดไว้
3. **Session ที่ยาวที่สุดของแต่ละวัน** — bar chart เห็นวันที่หลุดยาว
4. **Top 5 แอปตามเวลา** — donut หรือ bar
5. **Skip timeline** — จุดสีแดงเรียงตามเวลา เห็นว่าชอบข้ามช่วงไหน (มักจะเป็นบ่าย 3)
6. **Streak counter ใหญ่ๆ** — "พักครบติดต่อกัน 12 วัน"

เทคนิคทำให้ทำง่าย: แทนที่จะฝังกราฟใน Qt (ยุ่ง) ให้ generate เป็น **HTML ไฟล์เดียว** ด้วย Plotly/Chart.js แล้วเปิดใน browser หรือฝังใน `QWebEngineView` เขียนเร็วกว่ามาก สวยกว่า และ screenshot ลง portfolio ได้เลย

---

10. เทคโนโลยีที่แนะนำ (Python / Windows)

ส่วน	Library	หมายเหตุ
GUI + Overlay	**PySide6**	ทำ frameless/always-on-top/หลายจอ/animation ได้ครบ ใบอนุญาต LGPL ใช้ฟรี
Tray icon	`QSystemTrayIcon` (มากับ PySide6)	ไม่ต้องลง pystray เพิ่ม
Win32 API	`ctypes` (built-in) + `pywin32`	idle time, lock, foreground window
Process info	`psutil`	ชื่อ process จาก PID
Audio detection	`pycaw`	เช็คว่ามีเสียงเล่นอยู่
Database	`sqlite3` (built-in)	ไม่ต้องลงอะไร
Config	`pydantic` + JSON	validate ค่าอัตโนมัติ กัน config พัง
กราฟ	`plotly` → HTML	ง่ายและสวยกว่า matplotlib ในเคสนี้
แพ็ค .exe	`PyInstaller --noconsole`	ดูข้อควรระวังข้อ 11
Autostart	เขียน registry `HKCU\...\CurrentVersion\Run`	ง่ายกว่า Task Scheduler และไม่ต้อง admin

ทางเลือกถ้าอยากเบา: Tkinter ทำ overlay ได้ (`-fullscreen`, `-topmost`, `-alpha`) และมากับ Python เลย แต่พอต้องทำ animation ลื่นๆ กับหลายจอจะเริ่มเจ็บ แนะนำ PySide6 ตั้งแต่แรกถ้าคิดจะปล่อยให้คนอื่นใช้

---

11. กับดักที่ต้องรู้ล่วงหน้า (ประหยัดเวลาคุณได้หลายวัน)

1. **อย่าใช้ keyboard hook** — `GetLastInputInfo` ให้ผลเหมือนกันในเคสนี้ แต่ไม่โดน antivirus ตีเป็น keylogger และไม่ต้องขอสิทธิ์อะไรเลย

2. **PyInstaller + .exe ไม่มี signature = Windows Defender / SmartScreen เตือน** ถ้าจะปล่อยให้คนอื่นใช้จริง ต้องมี code signing certificate (~$100+/ปี) หรือ ปล่อยเป็น source + คำสั่งติดตั้งไปก่อน
3. **เกม fullscreen exclusive overlay ขึ้นไม่ได้** — เป็นข้อจำกัดของ Windows เอง ต้องยอมรับและเขียนใน docs ว่าโหมดนี้จะข้ามการเตือน (หรือใช้เสียงเตือนแทน)
4. **หลายมอนิเตอร์ = ต้องสร้าง overlay window ต่อจอ** ใช้ `QGuiApplication.screens()` และอย่าลืมเคสเสียบ/ถอดจอกลางคัน
5. **DPI scaling** — โน้ตบุ๊ค 150% กับจอนอก 100% ปนกัน ต้องตั้ง DPI awareness ไม่งั้น overlay จะเบลอหรือขนาดเพี้ยน
6. **เครื่อง sleep แล้วตื่นมาตัวนับต้องไม่พัง** — ใช้เทคนิคเทียบ timestamp gap ในข้อ 3.4
7. **ห้ามให้ปิดโปรแกรมง่ายๆ ผ่าน Task Manager แล้วจบ** — ถ้าจริงจัง ให้บันทึกว่า ถูกปิดกลางคัน และเตือนตอนเปิดครั้งหน้า (ทำแบบ "ให้เห็น" ดีกว่าทำแบบ "ห้าม" เพราะการพยายามฆ่าไม่ตายจะทำให้แอปดูเหมือนมัลแวร์)
8. **ทดสอบยาก** — ทำ "debug mode" ที่ย่อ 20 นาทีเหลือ 20 วินาทีตั้งแต่วันแรก ไม่งั้นคุณจะนั่งรอ 20 นาทีต่อการทดสอบหนึ่งครั้ง

---

12. Roadmap แบ่งเป็นเฟส

Phase 0 — Spike (1–2 วัน)

พิสูจน์ว่าทำได้: สคริปต์เดียว อ่าน idle time พิมพ์ออก console + เด้งหน้าต่างสีดำ เต็มจอเมื่อครบ 20 วินาที ยังไม่ต้องสวย

Phase 1 — MVP ใช้เองได้ (1 สัปดาห์)

- State machine + tick loop
- Overlay + นับถอยหลัง + toast เตือนล่วงหน้า
- Skip แบบกดค้าง 5 วินาที
- Tray icon (แสดงเวลาที่เหลือ / pause / exit)
- config.json แก้ด้วยมือได้
- **จุดหมาย: ใช้เองจริงสัก 1 สัปดาห์ แล้วจดว่าอะไรน่ารำคาญ** — ขั้นนี้สำคัญที่สุด

Phase 2 — ทำให้ทน (1–2 สัปดาห์)

- SQLite + บันทึกทุกเหตุการณ์
- Active window tracking + audio detection
- Whitelist + DND
- ระบบ Eye Debt
- Grace period (รอจังหวะที่ไม่ได้พิมพ์)
- Autostart

Phase 3 — ทำให้คนอื่นใช้ได้ (2–3 สัปดาห์)

- หน้า Settings เต็มรูปแบบ
- Dashboard + กราฟ
- กิจกรรมพักหลายแบบ
- Onboarding ครั้งแรก (ถามว่าทำงานกี่โมงถึงกี่โมง)
- Installer + README + screenshot
- ภาษาไทย/อังกฤษ

Phase 4 — ของเล่นเสริม (ถ้ายังสนุกอยู่)

- โหมดกล้อง: ตรวจระยะห่างจากจอ + นับอัตราการกระพริบตาจริง (ต่อยอดสวยมาก)
- Export รายงาน PDF รายสัปดาห์
- ปรับเวลาอัตโนมัติตามพฤติกรรม (ถ้าข้ามบ่อยช่วงบ่าย → ขยับเวลาพัก)
- ซิงก์กับมือถือ

---

13. โครงสร้างโฟลเดอร์ที่แนะนำ

```
blink/

├── blink/

│   ├── __main__.py          # entry point

│   ├── core/

│   │   ├── ticker.py        # loop 1 วินาที

│   │   ├── state.py         # state machine

│   │   ├── policy.py        # 20-20-20, escalation, debt

│   │   └── models.py        # dataclass ต่างๆ

│   ├── sensors/

│   │   ├── idle.py          # GetLastInputInfo

│   │   ├── window.py        # foreground window
```

```
│   │   ├── audio.py          # pycaw
│   │   └── session.py        # lock/sleep detection
│   ├── ui/
│   │   ├── overlay.py        # หน้าจอพัก
│   │   ├── toast.py          # เตือนล่วงหน้า
│   │   ├── tray.py
│   │   ├── settings.py
│   │   └── activities/       # กิจกรรมพักแต่ละแบบ
│   ├── storage/
│   │   ├── db.py
│   │   └── config.py
│   └── dashboard/
│       └── report.py         # generate HTML
├── tests/
│   └── test_policy.py        # ทดสอบ policy ด้วยเวลาปลอม (fake clock)
├── assets/
├── pyproject.toml
└── README.md
```

เคล็ดลับสำหรับ portfolio: ทำให้ `core/` ไม่มี import ของ Windows API หรือ Qt เลย รับเวลาและค่า sensor เข้ามาเป็น parameter → เขียน unit test ได้เต็มที่ด้วยเวลาปลอม "เร่งเวลา 8 ชั่วโมงใน 0.1 วินาที" ได้ อันนี้คนรีวิวโค้ดจะประทับใจมาก

---

14. มุมที่จะทำให้โปรเจคนี้โดดใน portfolio

- **ปัญหาที่คนเข้าใจได้ทันที** ไม่ต้องอธิบายเยอะ — ต่างจากพวก to-do app ที่มีเป็นล้าน
- **ระบบ Eye Debt** — เป็นดีไซน์เชิงพฤติกรรมที่คิดเอง มีเรื่องเล่า ไม่ใช่แค่ CRUD
- **Privacy-first by design** — ไม่มี network, ไม่ hook keyboard, hash window title เขียนเป็นหัวข้อหนึ่งใน README ได้เลย
- **มีข้อมูลจริงจากการใช้เอง** — screenshot Dashboard ของตัวเองหลังใช้ 1 เดือน พร้อมประโยคว่า "compliance rate ขึ้นจาก 30% เป็น 78%" นี่คือ proof ที่แข็งที่สุด
- **จัดการ edge case ของ OS ได้จริง** — sleep, หลายจอ, DPI, fullscreen แสดงว่าเข้าใจงาน desktop จริงๆ ไม่ใช่แค่ทำ demo
- **มี test ที่ทดสอบ logic เวลาได้** — หายากในโปรเจคนักศึกษา

---

15. ข้อสรุป 7 ข้อที่ค้างไว้ (ตัดสินใจแล้ว)

15.1 Overlay บัง taskbar ทั้งหมด แต่มีนาฬิกาของตัวเอง

บังหมด เพราะถ้าเว้น taskbar ไว้ คุณจะเห็น badge แจ้งเตือน Discord/Line เด้ง แล้วใจไม่อยู่กับการพักเลย — วัตถุประสงค์ของการพักคือ "ตัดขาด" ไม่ใช่ "หรี่ลง"

ทางเทคนิคก็ง่ายกว่าด้วย: `showFullScreen()` + `Qt.WindowStaysOnTopHint` ครอบทั้ง screen geometry จบ ไม่ต้องไปคำนวณ `availableGeometry()` แล้วเจอปัญหา taskbar auto-hide หรือ taskbar อยู่ด้านซ้าย

ชดเชยด้วย: นาฬิกาจริงตัวใหญ่มุมขวาบนของ overlay + ถ้ามี notification สำคัญ ค้างอยู่ ให้ Windows เด้งเสียงตามปกติ (เราไม่ได้ปิดเสียงระบบ)

15.2 ปุ่มฉุกเฉิน — มี และออกแบบให้เป็น "สัญญาณ" ไม่ใช่ "ช่องโหว่"

นี่คือรายละเอียดที่ต้องระวังที่สุด เพราะถ้าออกแบบพลาด **ทุกคนจะกดปุ่มฉุกเฉินแทน ปุ่ม Skip ตลอดเวลา** แล้วระบบต้นทุนทั้งหมดจะพัง

	ปุ่ม Skip	**ปุ่มฉุกเฉิน**
ตำแหน่ง	มุมล่างขวา	มุมล่างซ้าย (คนละฝั่ง กดพลาดยาก)
ต้นทุนทันที	กดค้าง 5 วินาที	ไม่มี กดครั้งเดียวปิดเลย
หนี้สายตา	+20 วินาที	ไม่มี

	ปุ่ม Skip	ปุ่มฉุกเฉิน
การบันทึก	นับเป็น skip ธรรมดา	บันทึกแยกประเภท `emergency`
ข้อความบนปุ่ม	"ข้ามรอบนี้"	"ฉุกเฉิน — ปิดทันที (จะถูกบันทึก)"

กลไกที่ทำให้ไม่ถูกใช้พร่ำเพรื่อ — ไม่ใช่การลงโทษ แต่เป็นการถาม:

ถ้าใช้ปุ่มฉุกเฉินเกิน 3 ครั้งใน 7 วัน ให้ tray เด้งข้อความว่า

> "สัปดาห์นี้คุณกดฉุกเฉิน 5 ครั้ง ส่วนใหญ่ช่วง 14:00–16:00 ตั้งค่าให้ช่วงนี้พักถี่น้อยลง หรือเพิ่มช่วง DND ไหม?"

การใช้ปุ่มฉุกเฉินบ่อย **ไม่ได้แปลว่าผู้ใช้ไม่มีวินัย แต่แปลว่าเราตั้งค่าไม่เข้ากับ ชีวิตจริงของเขา** — ให้ระบบเรียนรู้จากตรงนี้แทนที่จะดุ อันนี้เป็นอีกจุดที่เขียนใน portfolio ได้ดี เพราะมันคือการมองปุ่ม escape hatch เป็น telemetry

เพิ่มเติม: ให้มีคีย์ลัด `Ctrl+Alt+Shift+Esc` ทำงานเหมือนปุ่มฉุกเฉิน เผื่อกรณีเมาส์ไม่ทำงานหรือ overlay ค้าง — และเป็น safety net ให้ตัวคุณเองตอน dev

15.3 พักยาว — เอา และให้เป็นจุดที่ "ล็อคจอจริง" ที่เดียว

ตอนแรกคุณตั้งใจทำ "โปรแกรมล็อคจอ" — ผมว่าตรงนี้แหละคือที่ที่การล็อคจริงสมเหตุสมผล

ประเภท	**ทุก**	**นาน**	**วิธีบังคับ**
พักสั้น (สายตา)	20 นาที	20 วินาที	Overlay
พักยาว (ร่างกาย)	**2 ชั่วโมง**	**5 นาที**	**`LockWorkStation()` ล็อคจอจริง**

เหตุผลที่พักยาวควรล็อคจริง แต่พักสั้นไม่ควร:

- 20 วินาที → ถ้าล็อคจอจริง คุณจะเสียเวลาพิมพ์รหัสผ่านนานพอๆ กับเวลาพัก = ไร้สาระ
- 5 นาที → การต้องพิมพ์รหัสผ่านใหม่เป็นต้นทุนที่สมเหตุสมผล และมันบังคับให้ **ลุกออกจากเก้าอี้จริงๆ** ไม่ใช่แค่นั่งจ้องจอดำ

รายละเอียดที่ห้ามพลาด: ก่อนล็อคต้องเตือนล่วงหน้า **60 วินาที** แบบชัดเจน (นับถอยหลังตัวใหญ่) เพราะการโดนล็อคจอโดยไม่ทันเซฟงานคือเหตุผลที่คนจะถอนแอปทิ้ง และในหน้าเตือนต้องมีปุ่ม "เลื่อน 5 นาที" ที่ใช้ได้ 2 ครั้งต่อรอบ

Default: เปิดพักยาวไว้ แต่ให้ปิดได้ในหน้าตั้งค่า และให้ตั้งได้ว่าจะ "ล็อคจอจริง" หรือ "แค่ overlay 5 นาที" สำหรับคนที่ไม่ได้ตั้งรหัสผ่าน Windows

15.4 เส้นแบ่งวัน — ให้โปรแกรมหาเอง

ค่าเริ่มต้น 04:00 น. (คนนอนดึกถึงตี 3 ยังนับเป็นวันเดิม ซึ่งถูกต้องตามความรู้สึก)

หลังใช้ไป 2 สัปดาห์ ให้คำนวณเอง: ดูข้อมูลย้อนหลัง หาช่วงเวลาต่อเนื่องที่ยาวที่สุด ที่ไม่มี activity เลย (= ช่วงที่คุณนอน) แล้วตั้งเส้นแบ่งวันไว้ **ตรงกลาง** ของช่วงนั้น ถ้าคุณนอน 01:00–08:00 เส้นแบ่งวันจะได้ 04:30 อัตโนมัติ

แล้วแจ้งผู้ใช้ว่า "ปรับเส้นแบ่งวันเป็น 04:30 ตามเวลานอนของคุณ" พร้อมปุ่มแก้เอง — ฟีเจอร์เล็กแต่ดูฉลาดมากเวลาสาธิต

เคสที่ต้องจัดการด้วย: เปิดคอมทิ้งไว้ข้ามคืนโดยไม่มี input เลย → เวลาช่วงนั้น ต้องไม่ถูกนับเป็นเวลาจ้องจอ (จัดการโดย IDLE state ในข้อ 4 อยู่แล้ว)

15.5 Dashboard — generate HTML แล้วเปิดใน browser

เลือกทางนี้ด้วยเหตุผลเชิงเทคนิคที่ชัดเจน:

- ถ้าฝัง `QWebEngineView` → ไฟล์ .exe จะบวมจาก ~40 MB เป็น **200+ MB** เพราะต้องแพ็ค Chromium ทั้งตัวไปด้วย ไม่คุ้มเลยสำหรับหน้าที่เปิดสัปดาห์ละครั้ง
- ถ้าวาดกราฟด้วย Qt เอง (`QtCharts` / `pyqtgraph`) → เขียนนานกว่ามาก และสวยสู้ Plotly ไม่ได้
- เปิดใน browser → **screenshot ลง portfolio ได้สวยทันที** ไม่มีขอบหน้าต่างแอป

การทำงาน: คลิก tray → "ดูสถิติ" → query SQLite → render Jinja2 template เป็น HTML ไฟล์เดียว (ฝัง Plotly.js inline) → `webbrowser.open()`

โบนัส: ไฟล์ HTML เดียวจบแบบนี้ ส่งให้คนอื่นดูได้เลย และเป็นฐานสำหรับฟีเจอร์ "export รายงานรายสัปดาห์" ในเฟส 4 โดยไม่ต้องเขียนอะไรเพิ่ม

15.6 สองภาษา ไทย + อังกฤษ — ใช้ JSON ธรรมดา อย่าใช้ Qt Linguist

อย่าใช้ระบบ `.ts`/`.qm` ของ Qt สำหรับโปรเจคขนาดนี้ มันต้องคอมไพล์ทุกครั้งที่แก้ ข้อความ และ tooling ยุ่งเกินความจำเป็นสำหรับแค่ 2 ภาษา

ใช้ `locales/th.json` กับ `locales/en.json` เป็น key-value ธรรมดา + ฟังก์ชัน `t("key")` เปลี่ยนภาษาได้ทันทีโดยไม่ต้องรีสตาร์ท

สามเรื่องที่คนมักพลาดเวลาทำภาษาไทย:

1. **ฟอนต์** — อย่าพึ่ง default font ของ Qt ให้ระบุ `Sarabun` หรือ `Noto Sans Thai` (ฝังไฟล์ฟอนต์ไปกับแอปเลย ปลอดภัยกว่าหวังว่าเครื่องปลายทางจะมี) และเพิ่ม `line-height` ขึ้นอีก ~15% เพราะสระบน/ล่างของไทยจะชนกันถ้าใช้ค่า default
2. **ความยาวข้อความ** — ไทยยาวกว่าอังกฤษราว 20–30% อย่า fix ความกว้างปุ่ม ให้ใช้ layout ที่ยืดได้ ไม่งั้นข้อความไทยจะโดนตัด
3. **การตัดคำ** — ไทยไม่มีเว้นวรรคระหว่างคำ Qt จะตัดบรรทัดมั่วได้ ถ้าข้อความไหนยาว ให้เขียนโดยใส่เว้นวรรคในจุดที่ตัดได้เอง

เลือกภาษาครั้งแรก: อ่านจาก `locale.getdefaultlocale()` ถ้าเป็น `th_TH` ใช้ไทย ไม่งั้นอังกฤษ แล้วให้เปลี่ยนได้ในหน้าตั้งค่า

สำหรับ portfolio: README เขียนอังกฤษเป็นหลัก มี `README.th.md` แยก และ screenshot ใช้ภาษาอังกฤษ เพื่อให้คนดูได้กว้างที่สุด

15.7 จอเดียวก่อน — แต่เขียนโค้ดเผื่อหลายจอตั้งแต่วันแรก

เห็นด้วยกับการโฟกัสจอเดียว **แต่มีข้อเดียวที่ขอให้ทำตั้งแต่แรก** เพราะมันแค่ 3–4 บรรทัด และถ้าไม่ทำจะต้องรื้อทั้งชั้น UI ทีหลัง:

ให้ `OverlayManager` เก็บ overlay เป็น **list** และสร้างจากการวน `QGuiApplication.screens()` ตั้งแต่ต้น — จอเดียวก็คือ list ที่มีสมาชิกตัวเดียว โครงสร้างไม่ต่างกันเลย แต่วันที่คุณต่อจอที่สองมันจะทำงานถูกทันที

ที่ยังไม่ต้องทำในเฟสแรก (ยากจริง เก็บไว้เฟส 3):

- ดักเหตุการณ์เสียบ/ถอดจอกลางคัน (`screenAdded` / `screenRemoved`)
- จอต่างขนาด/ต่าง DPI ต้องคำนวณขนาดฟอนต์แยกกัน
- ตัดสินใจว่าจะเอา "จอไหนเป็นจอหลัก" สำหรับแสดงตัวนับถอยหลัง

สิ่งเดียวที่ต้องทำวันแรกจริงๆ: ตั้ง DPI awareness ให้ถูกตั้งแต่ก่อนสร้าง `QApplication` ไม่งั้นบนโน้ตบุ๊ค scaling 125–150% overlay จะเบลอหรือขนาดเพี้ยน ซึ่งเป็นบั๊กที่หาสาเหตุยากมากถ้าไปเจอทีหลัง

---

16. ผลกระทบต่อ Roadmap หลังตัดสินใจ

สิ่งที่ต้องขยับจากแผนเดิมในข้อ 12:

เฟส	เพิ่มเข้ามา
Phase 1 (MVP)	ปุ่มฉุกเฉิน + คีย์ลัด, OverlayManager แบบ list, DPI awareness

เฟส	เพิ่มเข้ามา
Phase 2	พักยาว 2 ชม. + ล็อคจอจริง + เตือนล่วงหน้า 60 วิ, เส้นแบ่งวันแบบ fix 04:00
Phase 3	Dashboard HTML + Plotly, ระบบ 2 ภาษา, เส้นแบ่งวันแบบคำนวณเอง, ข้อความชวนปรับค่าเมื่อกดฉุกเฉินบ่อย
Phase 4	รองรับหลายจอเต็มรูปแบบ, โหมดกล้อง

เพิ่ม table ใน DB อีกหนึ่งช่อง:

-- แก้ breaks.outcome ให้รองรับ

-- 'completed' | 'skipped' | 'emergency' | 'deferred'

-- และเพิ่ม

breaks.break_type   -- 'short' | 'long'